GULZARSOFT

INTERNSHIP PROGRAM

Gen AI & Agentic AI Developer Roadmap

9-Week Intensive Project-Based Learning Program

LangChain | LangGraph | MCP | LlamaIndex

CrewAI | LiveKit | Production Deployment

Duration: 9 Weeks | 5 Days/Week | 8-9 Hours/Day

Prepared by: **Zain Ali**

Comprehensive Learning Path for Next-Generation AI Developers

Table of Contents

Program Overview

Week 1: Python & LLM Foundations

Project: Smart Document Q&A Bot

Week 2: LangChain Fundamentals

Project: RAG Knowledge Base System

Week 3: LangGraph & Agent Orchestration

Project: Multi-Agent Research Assistant

Week 4: LlamaIndex & Advanced RAG

Project: Enterprise Document Intelligence

Week 5: CrewAI & Multi-Agent Teams

Project: Automated Content Creation Team

Week 6: MCP & Tool Ecosystem

Project: Universal Tool Agent

Week 7: LiveKit & Real-Time AI

Project: Voice AI Assistant

Week 8: Production & Deployment

Project: Production AI Platform

Week 9: Capstone Project

Project: Full-Stack Agentic AI Application

Additional Resources

Program Overview

Program Structure

This 9-week intensive program is designed to transform a developer into a proficient Gen AI & Agentic AI engineer. Each week follows a project-based learning approach where theoretical concepts are immediately applied to build real-world applications.

Weekly Structure

- 5 working days per week
- 8-9 hours daily commitment
- Morning: Theory & Concepts
- Afternoon: Hands-on Implementation
- Friday: Project completion & review

Learning Methodology

- Learn by building approach
- Real-world project scenarios
- Code reviews & feedback
- Documentation & best practices
- Portfolio-ready projects

Technologies Covered

Category	Technologies
LLM Frameworks	LangChain LangGraph LlamaIndex
Agent Frameworks	CrewAI LangGraph Agents AutoGen
Protocols & APIs	MCP OpenAI API Anthropic API
Real-Time AI	LiveKit WebRTC Voice AI
Deployment	FastAPI Docker AWS/GCP

Prerequisites

- Strong Python programming fundamentals
- Basic understanding of REST APIs
- Familiarity with Git and version control
- Basic knowledge of HTTP and web concepts
- Understanding of async/await patterns (helpful but not required)

Program Outcomes

By the end of this program, the intern will:

- Build production-ready RAG systems with multiple frameworks
- Design and implement multi-agent orchestration workflows
- Create real-time voice AI applications
- Deploy scalable AI systems to cloud infrastructure
- Have a portfolio of 9 complete projects demonstrating expertise

WEEK 1

Python & LLM Foundations

Capstone Project: ***Smart Document Q&A Bot***

Learning Objectives

- Master async Python patterns for AI applications
- Understand LLM fundamentals and API integration
- Learn prompt engineering best practices
- Build foundational knowledge for advanced frameworks

Daily Schedule

1 Python Refresher & Environment Setup

8 hours

THEORY (3 HOURS)

- Advanced Python concepts: decorators, generators, context managers
- Type hints and Pydantic for data validation
- Environment management: conda, poetry, virtualenv
- Project structure best practices for AI applications

PRACTICE (5 HOURS)

- Set up development environment with Python 3.11+
- Create project template with proper structure
- Implement custom decorators for logging and timing
- Build Pydantic models for API request/response validation

2 Async Programming & API Handling

8 hours

THEORY (3 HOURS)

- Async/await fundamentals and event loops
- HTTP clients: httpx vs aiohttp vs requests
- Rate limiting and retry strategies
- Error handling in async contexts

PRACTICE (5 HOURS)

- Build async HTTP client wrapper with retry logic
- Implement semaphore-based rate limiting
- Create batch processing system for API calls
- Handle timeouts, connection errors gracefully

3 OpenAI API & LLM Basics

9 hours

THEORY (3 HOURS)

- LLM architecture overview (transformers, attention)
- OpenAI API: chat completions, embeddings, function calling
- Tokenization and context window management
- Model selection: GPT-4, GPT-3.5, cost considerations

PRACTICE (6 HOURS)

- Integrate OpenAI API with async client
- Build streaming response handler
- Implement token counting and cost tracking
- Create basic chat completion wrapper class

4 Prompt Engineering Fundamentals

9 hours

THEORY (3 HOURS)

- Prompt structure: system, user, assistant messages
- Zero-shot, few-shot, and chain-of-thought prompting
- Output formatting: JSON mode, structured outputs
- Prompt versioning and A/B testing concepts

PRACTICE (6 HOURS)

- Build prompt template system with Jinja2
- Implement few-shot example selector
- Create structured output parsers
- Design prompts for different use cases (extraction, classification, generation)

5 Project: Smart Document Q&A Bot

9 hours

PROJECT IMPLEMENTATION

- Build CLI application for document Q&A
- Implement document loading (PDF, TXT, DOCX)
- Create chunking strategy for long documents
- Build simple in-memory vector search
- Integrate retrieval with LLM for answers
- Add conversation memory
- Write comprehensive README and documentation

Week 1 Deliverable: Smart Document Q&A Bot

- CLI application supporting PDF, TXT, DOCX uploads
- Document chunking with configurable size/overlap
- In-memory vector similarity search
- Context-aware Q&A with conversation history
- Streaming responses for better UX
- Cost tracking per query
- Unit tests with pytest

Evaluation Criteria

- Code quality and Python best practices
- Proper async implementation
- Error handling and edge cases
- Documentation completeness
- Test coverage (minimum 70%)

Resources & References

- Python AsyncIO Documentation
- OpenAI API Documentation
- Prompt Engineering Guide
- Pydantic Documentation
- Uvicorn & ASGI

WEEK 2

LangChain Fundamentals

*Capstone Project: **RAG Knowledge Base System***

Learning Objectives

- Master LangChain core abstractions and components

- Build production-grade RAG pipelines
- Implement vector stores and embeddings
- Understand chains, memory, and callbacks

1 LangChain Core Concepts

8 hours

THEORY (3 HOURS)

- LangChain architecture: Components, Chains, Agents
- Runnable interface and LCEL (LangChain Expression Language)
- Output parsers and structured outputs
- Chat models vs LLMs in LangChain

PRACTICE (5 HOURS)

- Set up LangChain with multiple providers (OpenAI, Anthropic)
- Build custom chains using LCEL
- Implement output parsers for different formats
- Create reusable chain components

2 Document Loaders & Text Splitters

8 hours

THEORY (3 HOURS)

- Document loaders: PDF, HTML, Markdown, databases
- Text splitting strategies: Character, Token, Recursive
- Chunk size optimization and overlap strategies
- Metadata preservation and enrichment

PRACTICE (5 HOURS)

- Implement custom document loaders
- Compare text splitter performance
- Build document processing pipeline
- Add metadata extraction (headers, source, date)

3 Embeddings & Vector Stores

9 hours

THEORY (3 HOURS)

- Embedding models: OpenAI, HuggingFace, Cohere
- Vector databases: Chroma, Pinecone, Weaviate, Qdrant
- Similarity search: cosine, dot product, Euclidean
- Hybrid search and reranking concepts

PRACTICE (6 HOURS)

- Set up ChromaDB and Pinecone
- Compare embedding model performance
- Implement similarity search with filters
- Build vector store abstraction layer

4 Retrievers & Memory

9 hours

THEORY (3 HOURS)

- Retriever types: VectorStore, BM25, MultiQuery, Contextual Compression
- Memory types: Buffer, Summary, Vector, Entity
- Conversation chains and context management
- Callbacks and tracing with LangSmith

PRACTICE (6 HOURS)

- Implement advanced retriever strategies
- Build conversation memory system
- Set up LangSmith for tracing
- Create custom callbacks for logging

5 Project: RAG Knowledge Base System

9 hours

PROJECT IMPLEMENTATION

- Build FastAPI backend for RAG system
- Implement document ingestion pipeline
- Create multi-tenant vector store (namespace per user)
- Build conversational retrieval chain
- Add source citation in responses
- Implement query rewriting for better retrieval
- Add evaluation metrics (relevance, faithfulness)

Week 2 Deliverable: RAG Knowledge Base System

- FastAPI application with document upload endpoints
- Support for multiple file formats (PDF, DOCX, TXT, MD)
- ChromaDB/Pinecone integration with metadata filtering
- Conversational RAG with source citations
- LangSmith integration for tracing and monitoring
- API documentation with OpenAPI/Swagger
- Docker containerization

Resources & References

- LangChain Documentation
- LCEL Documentation
- ChromaDB Documentation
- Pinecone Learning Center
- LangSmith Documentation

WEEK 3

LangGraph & Agent Orchestration

*Capstone Project: **Multi-Agent Research Assistant***

Learning Objectives

- Understand stateful agent workflows with LangGraph
- Build complex multi-step reasoning systems
- Implement human-in-the-loop interactions
- Master persistence and checkpointing

1 LangGraph Fundamentals

8 hours

THEORY (3 HOURS)

- LangGraph vs LangChain: when to use each
- Graph structure: States, Nodes, Edges
- State management and TypedDict schemas
- Compilation and execution model

PRACTICE (5 HOURS)

- Build first LangGraph application
- Define state schemas with TypedDict
- Create nodes for different processing steps
- Implement conditional edges for branching

2 Advanced State Management

8 hours

THEORY (3 HOURS)

- Reducers and state updates
- Subgraphs and modular design
- Parallel execution with Send
- State validation with Pydantic

PRACTICE (5 HOURS)

- Build complex state schemas
- Implement custom reducers
- Create reusable subgraph components
- Add parallel processing nodes

3 Multi-Agent Workflows

9 hours

THEORY (3 HOURS)

- Agent design patterns: Supervisor, Hierarchical, Network
- Agent communication and message passing
- Tool calling and function execution
- ReAct pattern implementation

PRACTICE (6 HOURS)

- Build supervisor-agent architecture
- Implement agent-to-agent communication
- Create tool-calling agents
- Build ReAct reasoning agent

4 Persistence & Human-in-the-Loop

9 hours

THEORY (3 HOURS)

- Checkpointer interface and implementations
- SQLite, Postgres, Redis checkpointing
- Human-in-the-loop patterns
- Interrupt and resume workflows

PRACTICE (6 HOURS)

- Implement checkpointing with SQLite
- Add human approval checkpoints
- Build resume-from-checkpoint functionality
- Create time-travel debugging interface

5 Project: Multi-Agent Research Assistant

9 hours

PROJECT IMPLEMENTATION

- Build research orchestrator with multiple agents
- Web Search Agent: DuckDuckGo, SerpAPI integration
- Analysis Agent: synthesize and summarize findings
- Writing Agent: generate structured reports
- Fact-Check Agent: verify claims with sources
- Implement human approval for critical decisions
- Add persistence for long-running research tasks

Week 3 Deliverable: Multi-Agent Research Assistant

- LangGraph-based multi-agent system
- 4 specialized agents with distinct roles
- Web search integration (SerpAPI/DuckDuckGo)
- Structured report generation (Markdown/PDF)
- Human-in-the-loop approval points
- SQLite persistence with checkpointing
- Streamlit UI for interaction

Resources & References

- LangGraph Documentation
- LangGraph GitHub
- LangGraph Blog Posts
- LangChain Agents

WEEK 4

LlamaIndex & Advanced RAG

*Capstone Project: **Enterprise Document Intelligence System***

Learning Objectives

- Master LlamaIndex for advanced RAG applications
- Implement sophisticated indexing strategies
- Build agentic RAG systems
- Understand query planning and optimization

1 LlamaIndex Core Concepts

8 hours

THEORY (3 HOURS)

- LlamaIndex vs LangChain: complementary approaches
- Documents, Nodes, and Index structures
- Data connectors and loaders
- Response synthesis modes

PRACTICE (5 HOURS)

- Set up LlamaIndex with multiple LLM providers
- Create different index types (Vector, List, Tree, Keyword)
- Build document ingestion pipeline
- Implement custom node parsers

2 Query Engines & Retrievers

8 hours

THEORY (3 HOURS)

- Query engine types and use cases
- Custom retrievers and postprocessors
- Reranking strategies: Cohere, ColBERT
- Hybrid search combining dense and sparse

PRACTICE (5 HOURS)

- Build custom query engines
- Implement reranking pipeline
- Create hybrid search retriever
- Add query transformation (HyDE, multi-query)

3 Advanced RAG Techniques

9 hours

THEORY (3 HOURS)

- Sentence window retrieval
- Auto-merging retrieval
- Recursive retrieval and routing
- Metadata extraction and filtering

PRACTICE (6 HOURS)

- Implement sentence window retrieval
- Build auto-merging retriever
- Create hierarchical retrieval system
- Add automatic metadata extraction

4 LlamaIndex Agents

9 hours

THEORY (3 HOURS)

- Agent types: OpenAI, ReAct, Custom
- Tool abstractions in LlamaIndex
- Multi-document agents
- Router agents for query planning

PRACTICE (6 HOURS)

- Build ReAct agent with LlamaIndex tools
- Create multi-document query agent
- Implement router agent for query classification
- Add custom tool integration

5 Project: Enterprise Document Intelligence

9 hours

PROJECT IMPLEMENTATION

- Build enterprise-grade document processing system
- Implement multiple index types for different content
- Create hierarchical document navigation
- Build agentic query router
- Add table extraction and structured data Q&A
- Implement citation and source tracking
- Create evaluation pipeline with RAGAS metrics

Week 4 Deliverable: Enterprise Document Intelligence System

- LlamaIndex-based enterprise RAG system
- Multiple index strategies (Vector + Tree)
- Advanced retrieval (sentence window, auto-merge)
- Agentic query routing and planning
- Table and structured data extraction
- RAGAS evaluation metrics
- Comparison with LangChain implementation

Resources & References

- LlamaIndex Documentation
- Production RAG Guide
- RAGAS Evaluation
- LlamaIndex GitHub

WEEK 5

CrewAI & Multi-Agent Teams

*Capstone Project: **Automated Content Creation Team***

Learning Objectives

- Master CrewAI framework for agent teams
- Design role-based agent architectures
- Implement collaborative task workflows
- Build autonomous content generation pipelines

1 CrewAI Architecture

8 hours

THEORY (3 HOURS)

- CrewAI philosophy and design principles
- Core components: Agents, Tasks, Crews, Tools
- Process types: Sequential, Hierarchical
- Agent role definition and backstory

PRACTICE (5 HOURS)

- Set up CrewAI environment
- Create first agent with role and goal
- Define tasks with context dependencies
- Build sequential crew workflow

2 Advanced Agent Design

8 hours

THEORY (3 HOURS)

- Agent delegation and collaboration
- Memory types: Short-term, Long-term, Entity
- Custom tool creation and integration
- Output parsers and structured responses

PRACTICE (5 HOURS)

- Build agents with memory capabilities
- Implement custom tools for agents
- Create hierarchical crew structure
- Add output validation and parsing

3 Task Orchestration

9 hours

THEORY (3 HOURS)

- Task dependencies and context passing
- Async task execution
- Human feedback integration
- Error handling and retry strategies

PRACTICE (6 HOURS)

- Build complex task pipelines
- Implement context sharing between agents
- Add human-in-the-loop checkpoints
- Create error recovery mechanisms

4 Tools & Integrations

9 hours

THEORY (3 HOURS)

- Built-in tools: search, scraping, file operations
- LangChain tools integration
- Custom tool development patterns
- Tool caching and optimization

PRACTICE (6 HOURS)

- Integrate SerpAPI for web search
- Build custom scraping tools
- Create file I/O and processing tools
- Implement tool result caching

5 Project: Automated Content Creation Team

9 hours

PROJECT IMPLEMENTATION

- Build content creation crew with specialized agents
- Research Agent: topic research and outline creation
- Writer Agent: draft content based on research
- Editor Agent: review and improve content
- SEO Agent: optimize for search engines
- Publisher Agent: format and prepare for publishing
- Add human approval at key stages

Week 5 Deliverable: Automated Content Creation Team

- CrewAI-based multi-agent content pipeline
- 5 specialized agents with defined roles
- Research-to-publication workflow
- Web search and content scraping integration
- Human approval checkpoints
- Output in multiple formats (Markdown, HTML)
- Performance metrics and cost tracking

Resources & References

- CrewAI Documentation
- CrewAI GitHub
- CrewAI Examples
- LangChain Tools Integration

WEEK 6

MCP & Tool Ecosystem

*Capstone Project: **MCP-Powered Universal Tool Agent***

Learning Objectives

- Understand Model Context Protocol (MCP) architecture
- Build MCP servers and clients
- Integrate external tools via MCP
- Create extensible tool ecosystems

1 MCP Fundamentals

8 hours

THEORY (3 HOURS)

- MCP architecture and protocol design
- Servers, Clients, and Transports
- Tools, Resources, and Prompts
- JSON-RPC communication model

PRACTICE (5 HOURS)

- Set up MCP SDK and environment
- Create first MCP server
- Build MCP client connection
- Test tool invocation flow

2 Building MCP Servers

8 hours

THEORY (3 HOURS)

- Server lifecycle and initialization
- Tool registration and schema definition
- Resource exposure patterns
- Error handling and logging

PRACTICE (5 HOURS)

- Build calculator MCP server
- Create file system MCP server
- Implement weather API MCP server
- Add authentication to MCP server

3 MCP Clients & Integration

9 hours

THEORY (3 HOURS)

- Client architecture and connection management
- Tool discovery and dynamic loading
- Multi-server client patterns
- Integration with Claude Desktop and other clients

PRACTICE (6 HOURS)

- Build MCP client with multiple server support
- Implement tool discovery and caching
- Create LLM integration with MCP tools
- Test with Claude Desktop

4 Advanced MCP Patterns

9 hours

THEORY (3 HOURS)

- Progress reporting for long operations
- Resource subscriptions and notifications
- Prompt templates and arguments
- Sampling and LLM completions from servers

PRACTICE (6 HOURS)

- Add progress reporting to tools
- Implement resource subscriptions
- Create prompt template server
- Build sampling-enabled MCP server

5 Project: Universal Tool Agent

9 hours

PROJECT IMPLEMENTATION

- Build universal agent with MCP tool integration
- Create 5+ MCP servers (calculator, search, files, database, API)
- Implement dynamic tool discovery and registration
- Build agent that can use any MCP tool
- Add tool result caching and optimization
- Create tool usage analytics dashboard

Week 6 Deliverable: MCP-Powered Universal Tool Agent

- Universal agent with MCP client integration
- 5+ custom MCP servers for different domains
- Dynamic tool discovery from multiple servers
- Web search, file operations, database access via MCP
- Tool usage analytics and cost tracking
- Configuration management for MCP servers
- Documentation for extending with new servers

Resources & References

- MCP Official Documentation
- MCP GitHub Organization
- MCP Python SDK
- Official MCP Servers

WEEK 7

LiveKit & Real-Time AI

*Capstone Project: **Real-Time Voice AI Assistant***

Learning Objectives

- Master LiveKit for real-time communication
- Build voice AI applications
- Implement multimodal agents
- Understand WebRTC and streaming

1 LiveKit Fundamentals

8 hours

THEORY (3 HOURS)

- LiveKit architecture: Rooms, Participants, Tracks
- WebRTC basics and media streaming
- Server and client SDKs
- Authentication and token management

PRACTICE (5 HOURS)

- Set up LiveKit server (cloud or self-hosted)
- Create room and join as participant
- Implement audio/video track publishing
- Build basic video conferencing app

2 LiveKit Agents

8 hours

THEORY (3 HOURS)

- LiveKit Agents framework
- VoicePipeline and agent patterns
- STT (Speech-to-Text) integration
- TTS (Text-to-Speech) integration

PRACTICE (5 HOURS)

- Build first LiveKit agent
- Integrate Deepgram for STT
- Add ElevenLabs or Cartesia for TTS
- Create voice-to-voice pipeline

3 Multimodal Agents

9 hours

THEORY (3 HOURS)

- Multimodal LLMs: GPT-4V, Gemini, Claude
- Vision capabilities in agents
- Screen sharing and video analysis
- Function calling in voice contexts

PRACTICE (6 HOURS)

- Build multimodal agent with vision
- Implement screen share analysis
- Add function calling to voice agent
- Create visual question answering agent

4 Real-Time Streaming

9 hours

THEORY (3 HOURS)

- Streaming audio processing
- Latency optimization techniques
- Interruption handling
- Barge-in and VAD (Voice Activity Detection)

PRACTICE (6 HOURS)

- Optimize audio pipeline for low latency
- Implement interruption handling
- Add VAD for barge-in detection
- Create smooth conversation flow

5 Project: Real-Time Voice AI Assistant

9 hours

PROJECT IMPLEMENTATION

- Build complete voice AI assistant
- Integrate with RAG for knowledge Q&A
- Add function calling for actions
- Implement conversation memory
- Create web UI for interaction
- Add analytics and call logging

Week 7 Deliverable: Real-Time Voice AI Assistant

- LiveKit-based voice AI agent
- End-to-end voice pipeline (STT → LLM → TTS)
- RAG integration for knowledge queries
- Function calling for task execution
- Interruption and barge-in handling
- Web UI with React/Next.js
- Call recording and analytics

Resources & References

- LiveKit Documentation
- LiveKit Agents
- LiveKit Agents GitHub
- Deepgram STT
- ElevenLabs TTS

WEEK 8

Production & Deployment

*Capstone Project: **Production-Ready AI Platform***

Learning Objectives

- Build production-grade AI applications with FastAPI
- Master Docker containerization
- Deploy to cloud infrastructure
- Implement monitoring and observability

1 FastAPI for AI Applications

8 hours

THEORY (3 HOURS)

- FastAPI async patterns for LLM apps
- Streaming responses (SSE, WebSockets)
- Dependency injection and middleware
- Background tasks and job queues

PRACTICE (5 HOURS)

- Build FastAPI app with streaming endpoints
- Implement WebSocket for real-time chat
- Add authentication (JWT, API keys)
- Create background job processing

2 Docker & Containerization

8 hours

THEORY (3 HOURS)

- Docker fundamentals for Python apps
- Multi-stage builds for optimization
- Docker Compose for multi-service apps
- Best practices for AI/ML containers

PRACTICE (5 HOURS)

- Create optimized Dockerfile for FastAPI
- Build multi-service Docker Compose setup
- Add vector database (Chroma/Postgres)
- Implement health checks and logging

3 Cloud Deployment

9 hours

THEORY (3 HOURS)

- AWS/GCP deployment options
- ECS, EKS, Cloud Run, or EC2
- Load balancing and auto-scaling
- Environment management and secrets

PRACTICE (6 HOURS)

- Deploy to AWS ECS or GCP Cloud Run
- Set up CI/CD pipeline (GitHub Actions)
- Configure load balancer
- Manage secrets with AWS Secrets Manager

4 Monitoring & Observability

9 hours

THEORY (3 HOURS)

- Logging best practices for AI apps
- Metrics: latency, tokens, costs
- Tracing with OpenTelemetry
- Alerting and dashboards

PRACTICE (6 HOURS)

- Set up structured logging
- Implement custom metrics collection
- Integrate LangSmith/Phoenix tracing
- Create Grafana dashboard

5 Project: Production AI Platform

9 hours

PROJECT IMPLEMENTATION

- Deploy complete RAG system to cloud
- Set up auto-scaling based on load
- Implement comprehensive monitoring
- Add rate limiting and usage quotas
- Create admin dashboard for analytics
- Write deployment documentation

Week 8 Deliverable: Production-Ready AI Platform

- Cloud-deployed FastAPI application
- Docker containerization with Compose
- CI/CD pipeline with GitHub Actions
- Monitoring with Prometheus/Grafana
- Structured logging and tracing
- Auto-scaling configuration
- Complete deployment documentation

Resources & References

- FastAPI Documentation
- Docker Documentation
- AWS ECS Documentation
- Prometheus Documentation
- OpenTelemetry Documentation

WEEK 9

Capstone Project

Final Project: ***Full-Stack Agentic AI Application***

Learning Objectives

- Integrate all learned technologies
- Build a comprehensive agentic AI system
- Demonstrate production readiness
- Create portfolio-worthy project

1 Project Planning & Architecture

8 hours

PLANNING (4 HOURS)

- Define project scope and requirements
- Design system architecture
- Select technology stack
- Create project timeline and milestones

SETUP (4 HOURS)

- Set up project repository structure
- Initialize services and dependencies
- Create database schemas
- Set up development environment

2 Backend Development - Part 1

9 hours

CORE IMPLEMENTATION

- Build FastAPI backend with auth
- Implement LangGraph agent orchestration
- Create RAG pipeline with LlamaIndex
- Add CrewAI for specialized tasks
- Integrate MCP servers for tools

3 Backend Development - Part 2

9 hours

ADVANCED FEATURES

- Add LiveKit voice integration
- Implement real-time WebSocket updates
- Create background job processing
- Add comprehensive error handling
- Write unit and integration tests

4 Frontend & Integration

9 hours

UI DEVELOPMENT

- Build React/Next.js frontend
- Create chat interface with streaming
- Add voice chat component
- Implement document upload and management
- Create admin dashboard

5 Deployment & Documentation

9 hours

FINALIZATION

- Deploy to production cloud environment
- Set up monitoring and alerting
- Write comprehensive README
- Create API documentation
- Record demo video
- Prepare project presentation

Week 9 Deliverable: Full-Stack Agentic AI Application

- Production-deployed full-stack application
- LangGraph + CrewAI + LlamaIndex integration
- MCP tool ecosystem
- LiveKit voice capabilities
- React/Next.js frontend
- Comprehensive test coverage
- Complete documentation and demo

Final Evaluation Criteria

- Code quality and architecture
- Technology integration depth
- Production readiness
- Documentation completeness
- Presentation and demo quality
- Innovation and problem-solving

Additional Resources

Recommended Learning Platforms

Platform	Resources
Deeplearning.AI	LangChain courses, Prompt Engineering
LangChain Academy	Official LangGraph and LangChain tutorials
YouTube Channels	AI Jason, Sam Witteveen, James Briggs
GitHub	Official repositories and example projects

Essential Tools & Services

Category	Tools
LLM APIs	OpenAI, Anthropic, Google Gemini, Cohere
Vector Databases	Pinecone, ChromaDB, Weaviate, Qdrant
Cloud Platforms	AWS, GCP, Azure
Monitoring	LangSmith, Phoenix, Weights & Biases
Voice AI	Deepgram, ElevenLabs, Cartesia, AssemblyAI

Community & Support

- LangChain Discord: Active community for Q&A
- Reddit r/LangChain: Discussions and resources
- GitHub Discussions: Framework-specific help
- Stack Overflow: Technical questions

Program Completion Certificate Criteria:

- All 9 weekly projects completed and reviewed
- Final capstone project deployed to production
- Code quality meets industry standards
- Comprehensive documentation provided
- Successful project presentation